

System Structures — Chapter 2



叶冠华

g.ye@bupt.edu.cn

School of Computer Science (National Pilot Software Engineering School)



北京邮电大学

Review

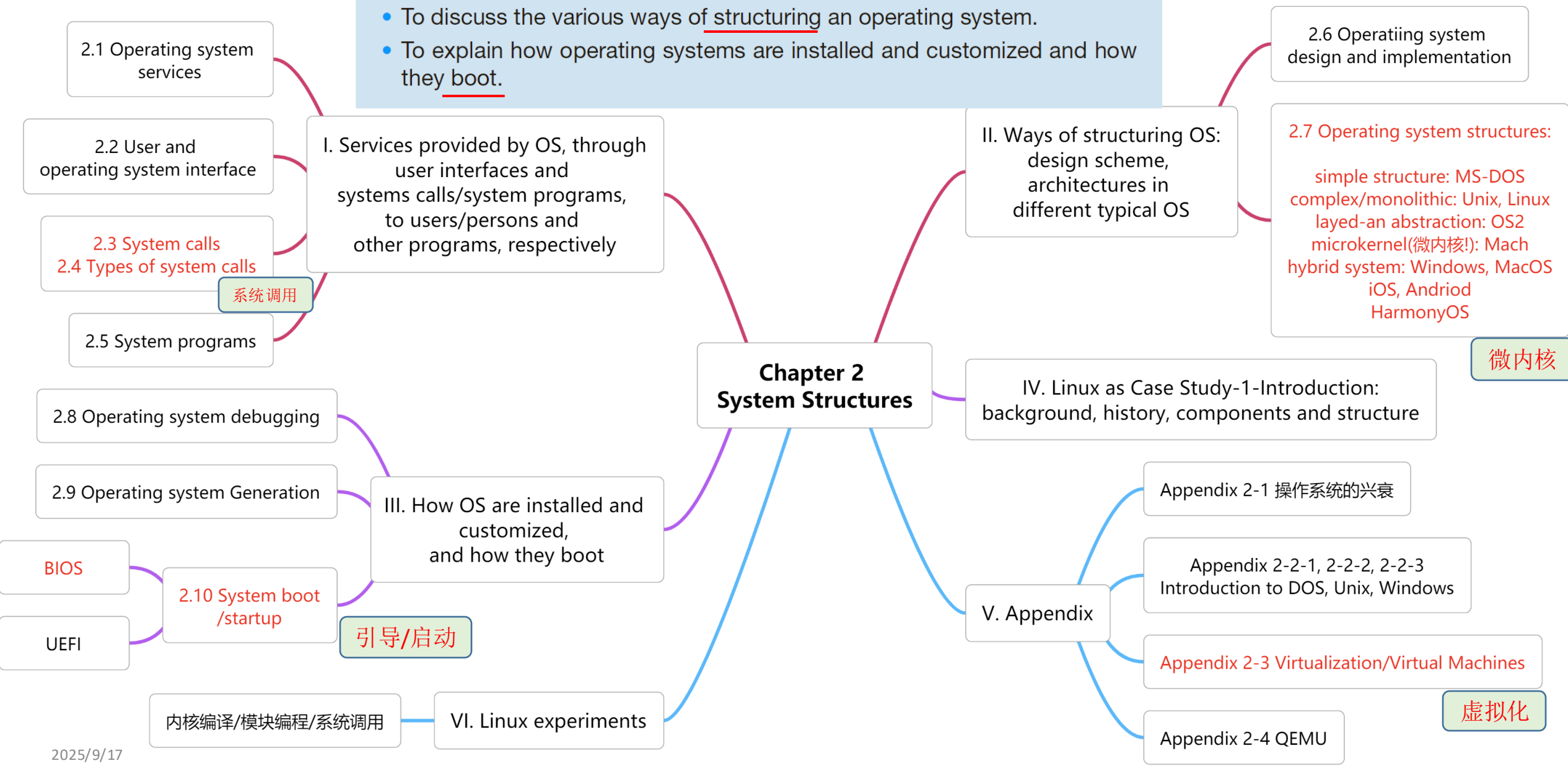
- 操作系统是计算机系统资源的管理者、
 - ▣ 处理机（CPU）管理、内存（Memory）管理、文件管理、设备管理
- 操作系统是用户和计算机硬件系统之间的接口
 - ▣ 命令接口（交互式/联机命令、批处理/脱机命令）
 - ▣ 程序接口（系统调用）
- 操作系统的特征
 - ▣ 并发Concurrency、共享Sharing、虚拟Virtualization、异步Asynchronism
 - ▣ 并发和共享是OS的两个最基本的基本特征、互为存在条件
- 操作系统的发展历程
 - ▣ 手工阶段→单道批处理OS→多道批处理OS→分时OS→现代OS
 - ▣ 单道批处理：内存存放单个进程，简单顺序队列
 - ▣ 多道批处理：内存存放多个进程，遇到I/O进程切换
 - ▣ 分时OS：内存存放多个进程，引入时间片

- 处理器运行模式
 - ▣ 用户态User Mode（不可执行特权指令）
 - ▣ 内核态Kernel Mode（可以执行特权指令）
- 中断（从用户态转到内核态的唯一途径）
 - ▣ 外中断（当前指令无关）：I/O完成信号、时钟中断等
 - ▣ 内中断（当前指令相关）：系统调用、除数为0、运算溢出、非法指令等

Outline

CHAPTER OBJECTIVES

- To describe the services an operating system provides to users, processes, and other systems.
- To discuss the various ways of structuring an operating system.
- To explain how operating systems are installed and customized and how they boot.

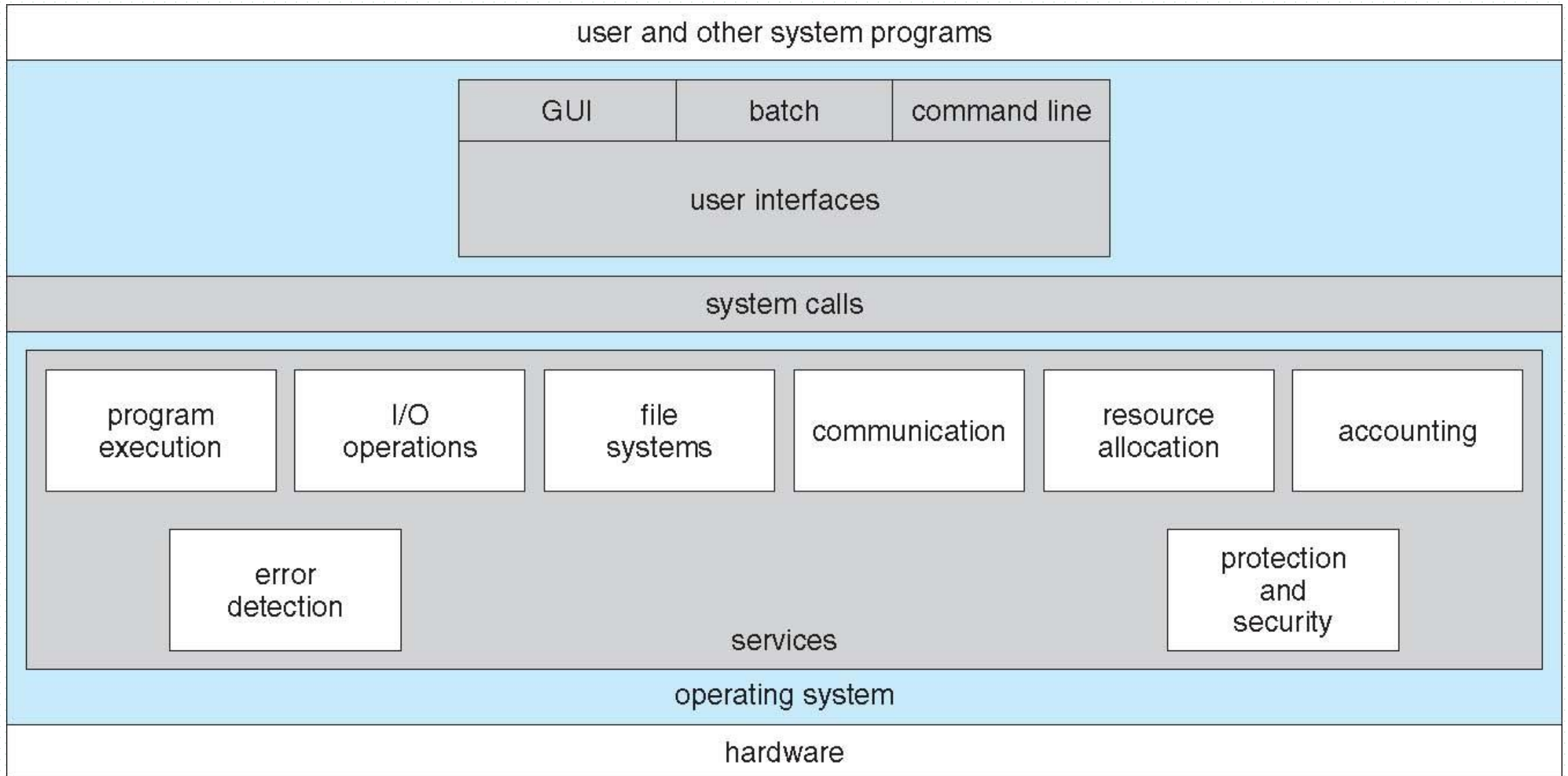


Terminology

User Interface (UI), Command-Line Interface (CLI), Graphics User Interface (GUI), Job control language(JCL)	用户接口, 命令行接口, 图形用户接口, 作业控制语言
command interpreter, shell	命令解释器, 壳
virtual reality, VR; augmented reality, AR; mixed reality, MR; extended reality, XR	虚拟现实, 增强现实, 混合现实, 扩展现实
message passing, shared-memory	消息传递, 共享内存
daemons	守护进程
monolithic/macro-kernel, micro-kernel	宏内核, 微内核

port	移植
emulation, simulation	仿真, 模拟
auditing, accounting, logging, registry	审计, 记账, 日志, 注册表
tuning, profiling	调优, 程序剖析
booting, bootstrap program, boot block, master boot record, MBR	引导, 引导程序, 引导块, 主引导记录

2.1 Operating Systems Services



Operating Systems Services

- Operating systems provide an environment for execution of programs and services to programs and users/persons
- One set of operating-system services provides functions that are helpful to the user:
 - ▣ **User interface** - Almost all operating systems have a **User Interface (UI)**.
 - Varies between **Command-Line Interface(CLI)/命令行接口**, **Graphics User Interface (GUI)**, **Batch/批处理(Job control language/作业控制语言, JCL)**
 - ▣ **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
 - ▣ **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
 - ▣ **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

Operating Systems Services

- ❑ **Communications** – Processes may exchange information, on the same computer or between computers over a network
 - Communications may be via shared memory or through message passing (packets moved by the OS)

- ❑ **Error detection** – OS needs to be constantly aware of possible errors
 - may occur in the CPU and memory hardware, in I/O devices, in user program
 - for each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Operating Systems Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
 - ▣ **resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
 - ▣ **accounting/记账** - To keep track of which users use how much and what kinds of computer resources
 - ▣ **protection and security/保护与安全** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - **protection** involves ensuring that all access to system resources is controlled
 - **security** of the system from outsiders requires user **authentication/认证**, extends to defending external I/O devices from invalid access attempts

2.2 User Operating System Interface

- Interaction between OS users and operating systems/computers
 - ▣ **persons as user**
 - CLI, GUI, touchscreen
 - voice/speech, gesture/body posture, brain-machine interface
 - ▣ **system and application programs**
 - system calls/API, Job Control Language JCL in batch systems
- CLI or **command interpreter/命令解释器** allows direct command entry
 - ▣ sometimes implemented in kernel
 - e.g. MS DOS (Disk Operating System)
 - ▣ sometimes by systems program, sometimes multiple flavors implemented – **shells**
 - interpreter + command programming language
 - Bourne shell, Bourne-Again shell (bash), zsh
 - ▣ primarily fetches a command from user and executes it
 - ▣ sometimes commands built-in, sometimes just names of programs
 - If the latter, adding new features doesn't require shell modification

Shell/外壳: 位于OS外层, 区别于内核, 提供用户界面的软—命令解释器(command interpreter);
(1) 负责将用户命令Command转化为OS可识别的指令;
(2) 也是一种程序设计语言。可交互式地解释和执行用户输入的命令, 也可以自动解释和执行预先设置好的一组命令

User Operating System Interface - GUI

- User-friendly **desktop** metaphor interface
 - ▣ usually mouse, keyboard, and monitor
 - ▣ **Icons/图标** represent files, programs, actions, etc
 - ▣ various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder/文件夹**))
 - ▣ invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
 - ▣ Microsoft Windows is GUI with CLI "command" shell
 - ▣ Apple Mac OS X is "Aqua" GUI interface with UNIX kernel underneath and shells available
 - ▣ Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)
 - 联机控制级窗口X-Windows for Linux

Touchscreen Interfaces

- Touchscreen devices require new interfaces
 - ▣ mouse not possible or not desired
 - ▣ actions and selection based on **gestures/姿态**
 - ▣ virtual keyboard for text entry
- 触觉传感器



Gesture/Posture-based Interfaces

- Gesture/pose(手势、姿态)
 - e.g. 手语识别与合成
 - 姿态识别方法
 - 数据手套
 - KINECT
 - 图像图像识别技术, 基于CV/DNN



微软Kinect:
动作感应游戏控制器

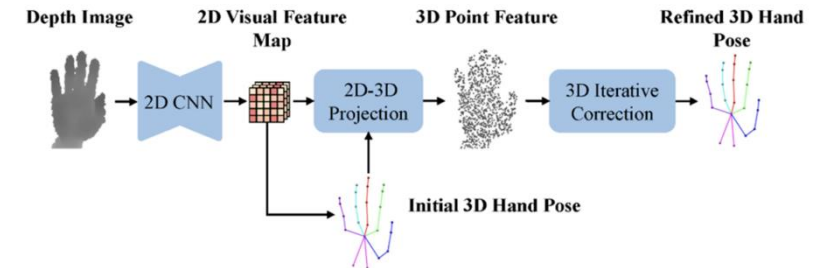
北京邮电大学师生首获人工智能顶会AAAI 2023杰出论文奖

北邮计语 2023-02-21 11:40 发表于北京

论文题目: 两个总比一个好: 用于三维手部姿态估计的图像点云网络

论文作者: 任鹏飞、陈宇琛、郝家畅、孙海峰、戚琦、王敬宇、廖建新

论文简介: 本文提出了一种图像点云网络(IPNet)用于精确和鲁棒的三维手部姿态估计。IPNet利用二维CNN高效地在二维图像空间中提取视觉表征, 并在三维点云空间中进行迭代校正, 挖掘深度数据的三维几何信息。特别地, 提出了一种基于稀疏锚点的“聚合-交互-传播”范式来细化点云特征, 减少了不规则的数据访问。同时, 在迭代修正过程中引入三维手模型先验信息, 显著提高了IPNet对遮挡和深度空洞的鲁棒性。



团队三维手部姿态估计成果曾获ICCV Hands竞赛榜单的第一名, 系列产品化成果荣获2019年、2021年互联网+创新创业大赛北京赛区两项一等奖。与字节跳动公司PICO团队合作, 研发手势控制算法成果应用于其产品线中, 实现在VR场景下的手势交互, 将有望成为人机交互的新方式。

Speech/Voice Interface

■ Speech/Voice

- 语音识别：将音频信号转换为符号信息，如语音录入
- 语音翻译：语音到文本的转换，如讲话口述系统
- 语音合成
- 语言理解
 - 识别语音，理解其含义

■ Voice commands, VUI

- e.g. 科大讯飞，小爱同学 in Xiaomi, based on speech recognition and NLP

Virtual Reality

■ VR, 虚拟现实

- ❑ the use of computer modeling and simulation that enables a person to interact with an artificial three-dimensional (3-D) visual or other sensory environment
- ❑ VR applications immerse the user in a computer-generated environment that simulates reality through the use of interactive devices, which send and receive information and are worn as goggles, headsets, gloves, or body suits
- ❑ 让用户沉浸其中的、由计算机生成的三维虚拟环境，并与现实环境相隔绝

■ AR (Augmented Reality, 增强现实)

- ❑ 在真实环境中增添或者移除由计算机实时生成的可以交互的虚拟物体或信息

■ MR (mixed reality, 混合现实)

- ❑ 通过全息图，将现实环境与虚拟环境相互混合，可以看成是VR与AR的混合

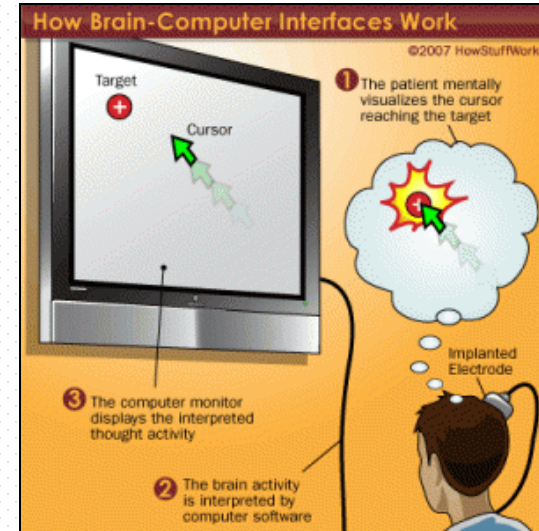
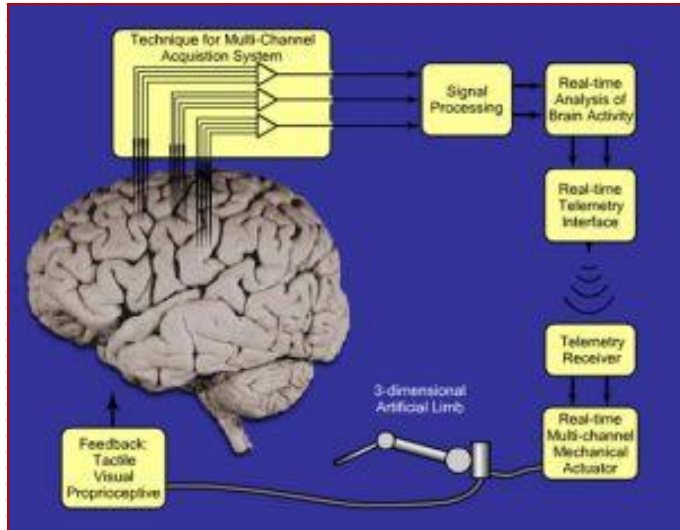


virtual reality headset

Human-Machine Interface

■ Brain-machine interface, BCI

- 实时记录人脑的脑电波，解读人的思维，并将其“翻译”成控制命令，实现对计算机、家用电器、机器人等设备的控制，从而做到“心想事成”
- 在医疗、康复、娱乐乃至国防领域有着广泛的应用前景
 - e.g. 助残，康复 【人文关怀】



2.3 System Call

- CPU kernel mode vs user mode
- **System calls** provide the programming interface to the services made available by the operating system, e.g. file access
 - ▣ 为允许用户态进程访问高权限资源，OS内核提供trap/system call
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level Application Program Interface (API) rather than direct system call use
- System call vs API in e.g. C/C++
 - ▣ read_sys() vs read()
- Three most common APIs
 - ▣ Win32 API for Windows
 - ▣ POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), Pthread API
 - ▣ Java API for the Java virtual machine (JVM)

Example

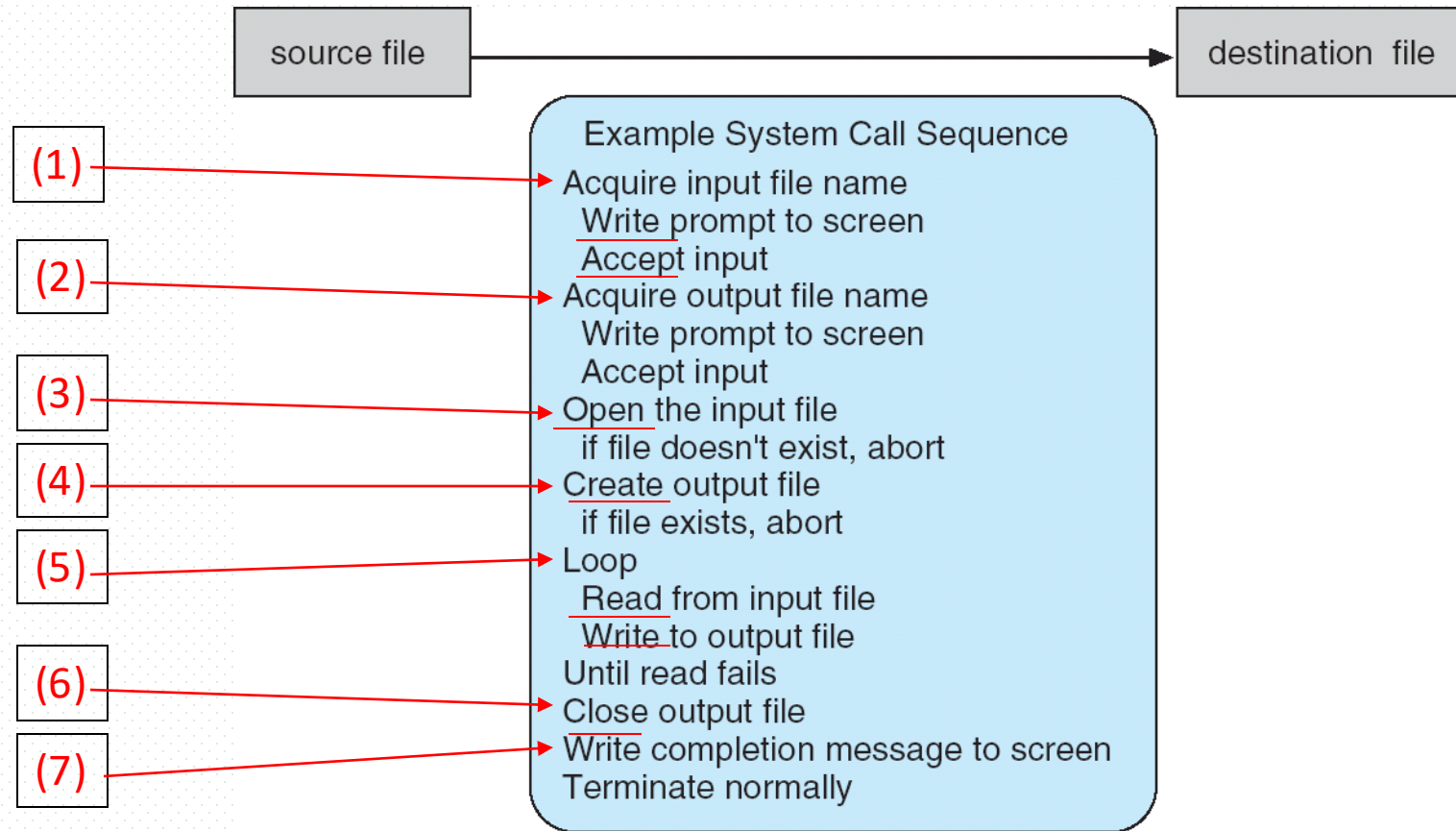


Fig. 2.1 System call to copy the contents of one file to another file

Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

<pre>#include <unistd.h></pre>		
<pre>ssize_t</pre>	<pre>read</pre>	<pre>(int fd, void *buf, size_t count)</pre>
return	function	parameters
value	name	

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

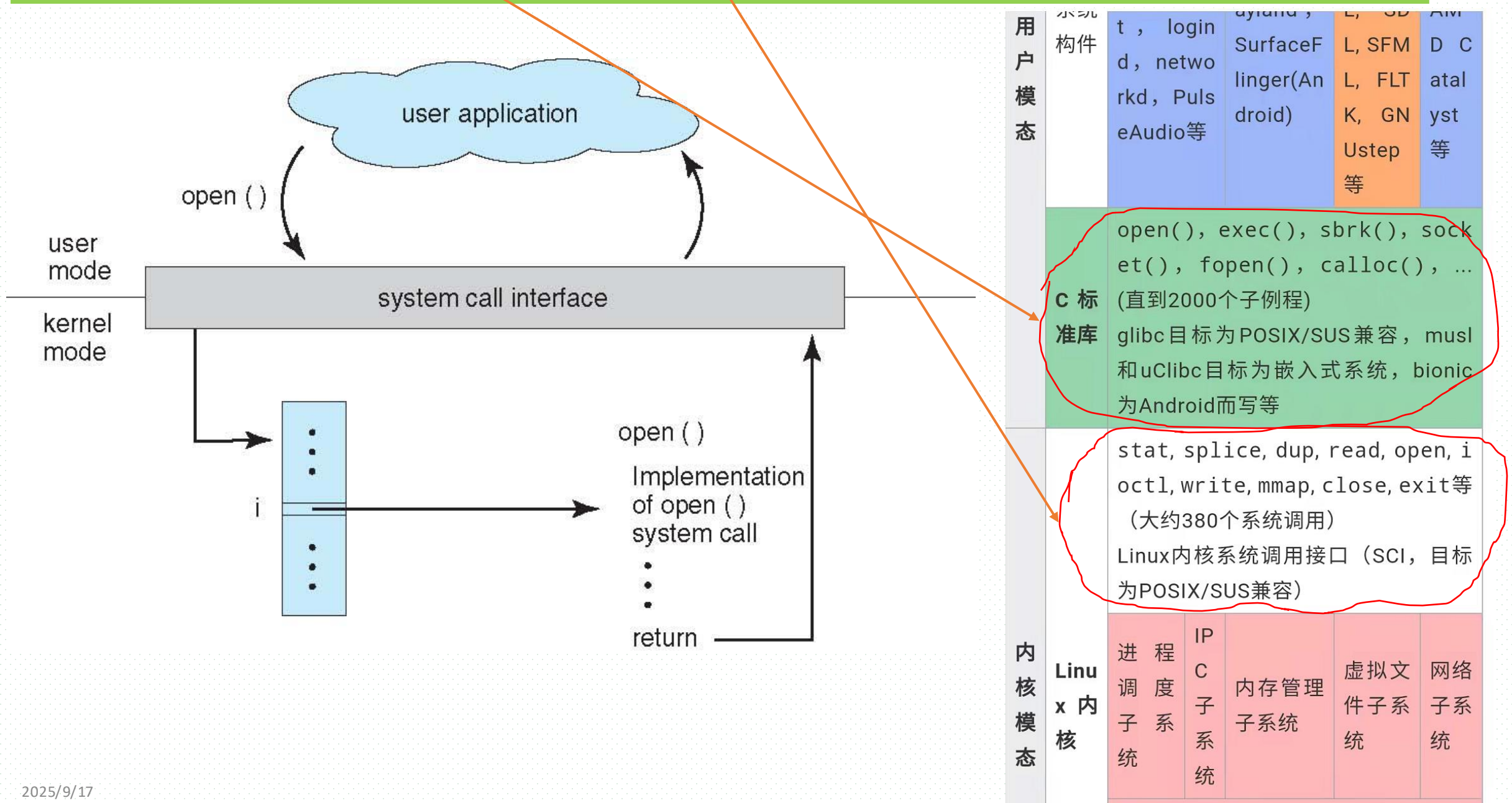
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

System Calls (Cont.)

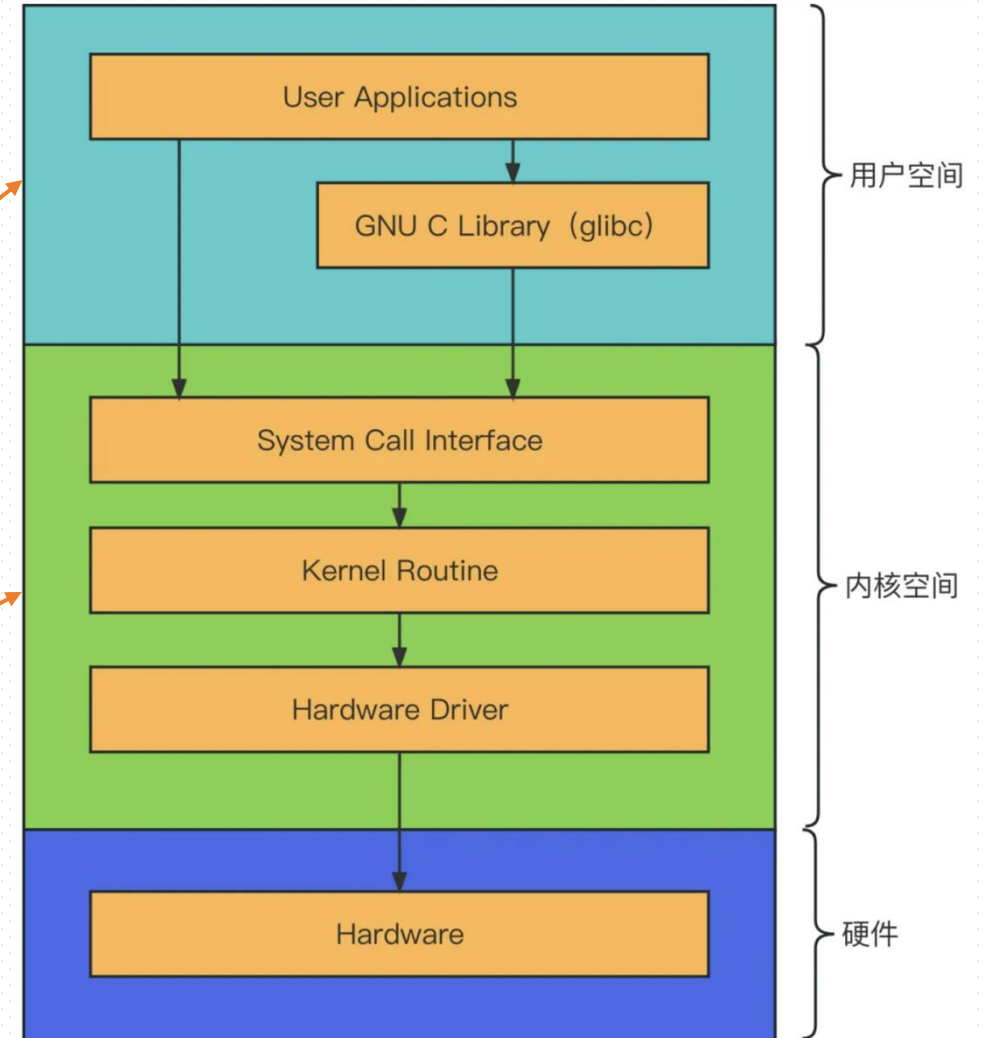
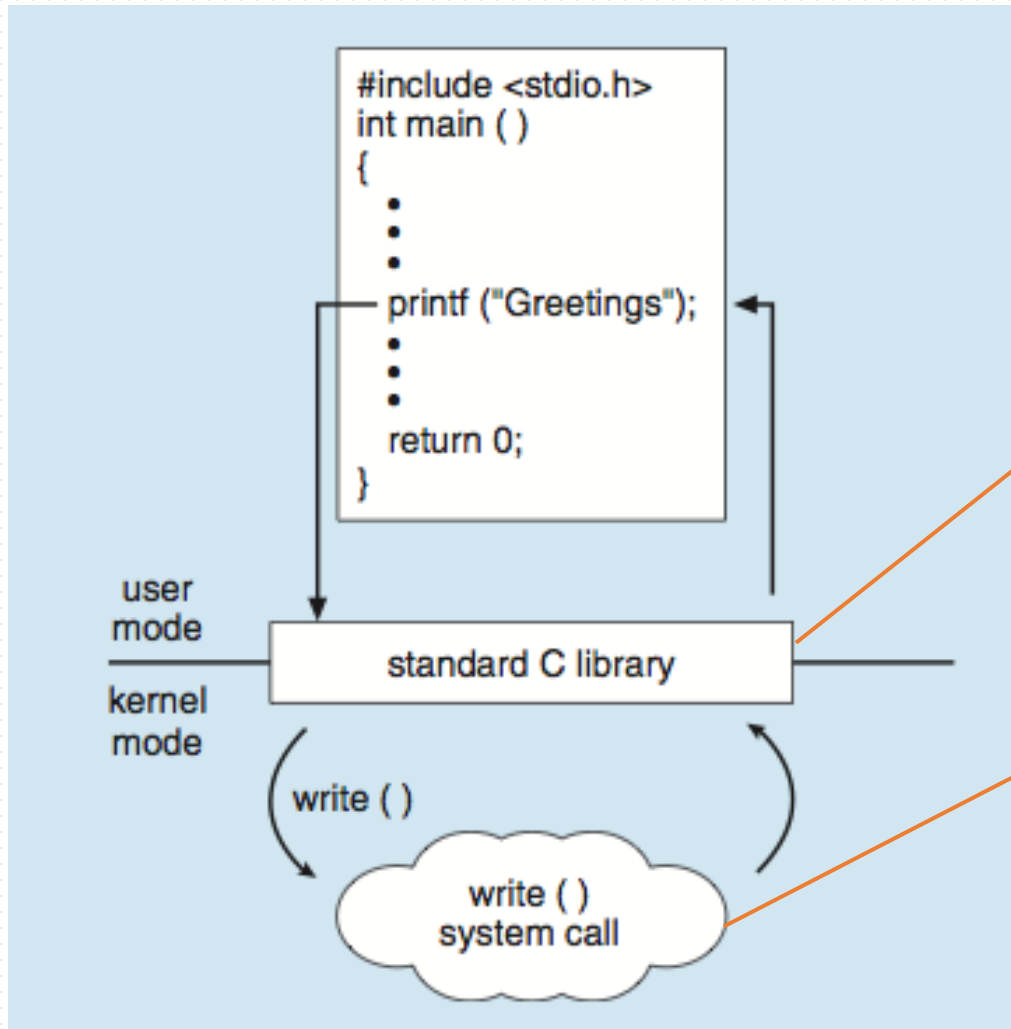
- Typically, a number associated with each system call (系统调用号)
 - ▣ the system-call interface maintains a table indexed according to these numbers
 - ▣ e.g.
 - in Linux, the system call numbers for **read()** and **write()** are **0** and **1**, respectively
 - Linux experiments, 向Linux内核中添加系统调用
- The **system call interface** invokes intended system call in OS kernel and returns status of the system call and any return values(Fig.2.3)
- The caller need know nothing about how the system call is implemented
 - ▣ just needs to obey API (Application Programming Interface, 应用编程接口) and understand what OS will do as a result call
 - ▣ most details of OS interface hidden from programmer by API
 - managed by run-time support library (set of functions built into libraries included with compiler)
 - e.g. `syscall()` in POSIX

API – System Call – OS Relationship



Standard C Library Example

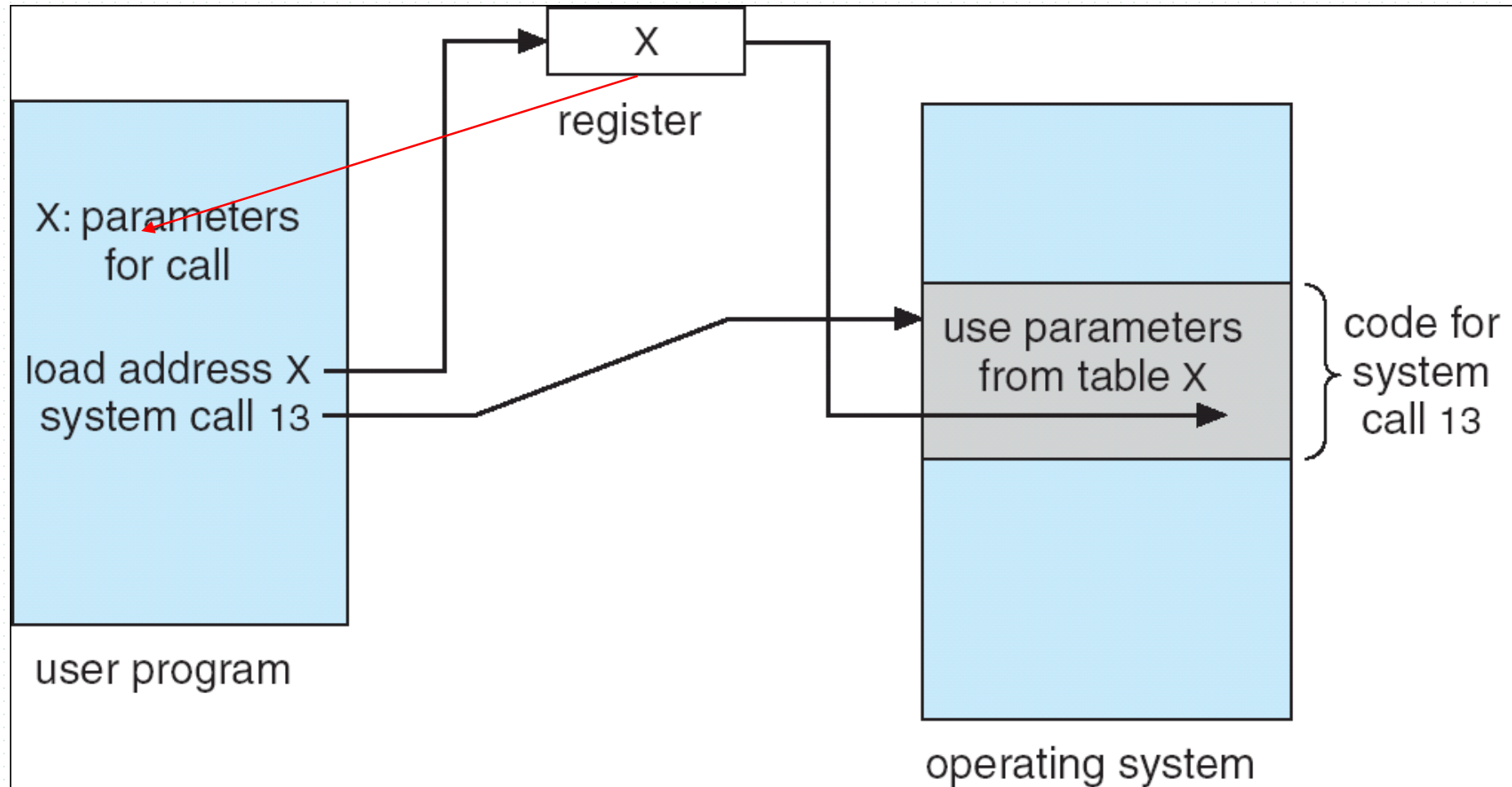
- API vs system call
 - ▣ C program invoking printf() library call, which calls write() system call



System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
 - ▣ Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
 - ▣ (寄存器传递) simplest: pass the parameters in **registers**
 - in some cases, may be more parameters than registers
 - ▣ (块/参数表传递) parameters stored in a **block**, or **table**, in memory, and address of block passed as a parameter in a register
 - this approach taken by Linux and Solaris
 - ▣ (栈传递) parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
 - ▣ block and stack methods do not limit the number or length of parameters being passed

Parameter Passing via Table



Parameter Passing in Linux

- x-86 32-bits/Linux环境下，system call参数传递方式
 - ▣ 寄存器传递方式
 - 参数个数不超过5个时，依次采用CPU的5个寄存器ebx, ecx, edx, esi, edi传递参数
 - ▣ 参数表传递方式
 - 参数个数超过5个时，将这些参数放在用户空间的参数表中，利用一个单独的寄存器记录参数表的地址指针，系统调用发生时，将参数表地址指针传递给内核
- 系统调用返回值也通过寄存器传递，存放在eax寄存器中

2.4 Types of System Calls

- Categories
 - ▣ process control
 - ▣ file management
 - ▣ device management
 - ▣ information maintenance
 - ▣ communications
 - ▣ protection

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

■ 系统调用表sys_call_table

系统调用号 ↔ 系统调用处理程序入口地址

- ❑ Linux内核数据结构，定义在[arch/x86/kernel/syscall_64.c](#)文件中
- ❑ 数组，内有__NR_syscall_max+1个元素
 - 参数__NR_syscall_max是1个宏，在64位模式下其值为542
 - 系统最多允许542+1=543个系统调用
- ❑ 数组中每个元素的类型是函数指针sys_call_ptr_t
 - 函数：系统调用处理函数
 - 指针：处理函数入口地址
- ❑ 每个系统调用对应表中1个系统调用，以系统调用号作为数组下标，记录系统调用的系统调用号与系统调用处理程序的入口地址

x86-64/Linux系统调用执行步骤

- Step1. 用户应用程序在用户空间（用户态下）将系统调用号及参数传入%rax等指定寄存器
- Step2. 使用syscall从用户态切换到内核态，从入口system_call开始执行
- Step3. 保存用户态栈指针，然后切换到内核栈；接着，将传参用通用寄存器、返回地址及系统调用号存入内核栈
- Step4. 检查系统调用号，检查通过后，在系统调用表sys_call_table里根据调用号找到对应的函数入口，执行函数调用；如果检查未通过，在%rax里放入错误码-ENOSYS直接返回
- Step5. 函数调用执行完毕后，恢复现场，包括通用寄存器、rflags寄存器，返回地址，栈指针
- Step6. 使用sysret从内核态返回到用户态，接着这些用户应用程序

2.5 System Programs

- System programs provide a convenient environment for program development and execution. They can be divided into:
 - ▣ File manipulation
 - ▣ Status information
 - ▣ File modification
 - ▣ Programming language support
 - ▣ Program loading and execution
 - ▣ Communications
 - ▣ Background services
 - ▣ Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls

System Programs (Cont.)

■ File management

- ❑ create, delete, copy, rename, print, dump, list, and generally manipulate files and directories

■ Status information

- ❑ some ask the system for info - date, time, amount of available memory, disk space, number of users
- ❑ others provide detailed performance, **logging/日志**, and debugging information
- ❑ typically, these programs format and print the output to the terminal or other output devices
- ❑ some systems implement a **registry/注册表** - used to store and retrieve configuration information

System Programs (cont.)

- **File modification**
 - ▣ Text editors to create and modify files
 - ▣ Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
 - ▣ Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

System Programs (cont.)

■ Background Services

- Launch at boot time
 - some for system startup, then terminate
 - some from system boot to shutdown
- Provide facilities like disk checking, process scheduling, error logging, printing
- Run in user context not kernel context
- Known as **services**, **subsystems**, **daemons/守护进程**

■ Application programs

- Don't pertain to system
- Run by users
- Not typically considered as part of OS
- Launched by command line, mouse click, finger poke

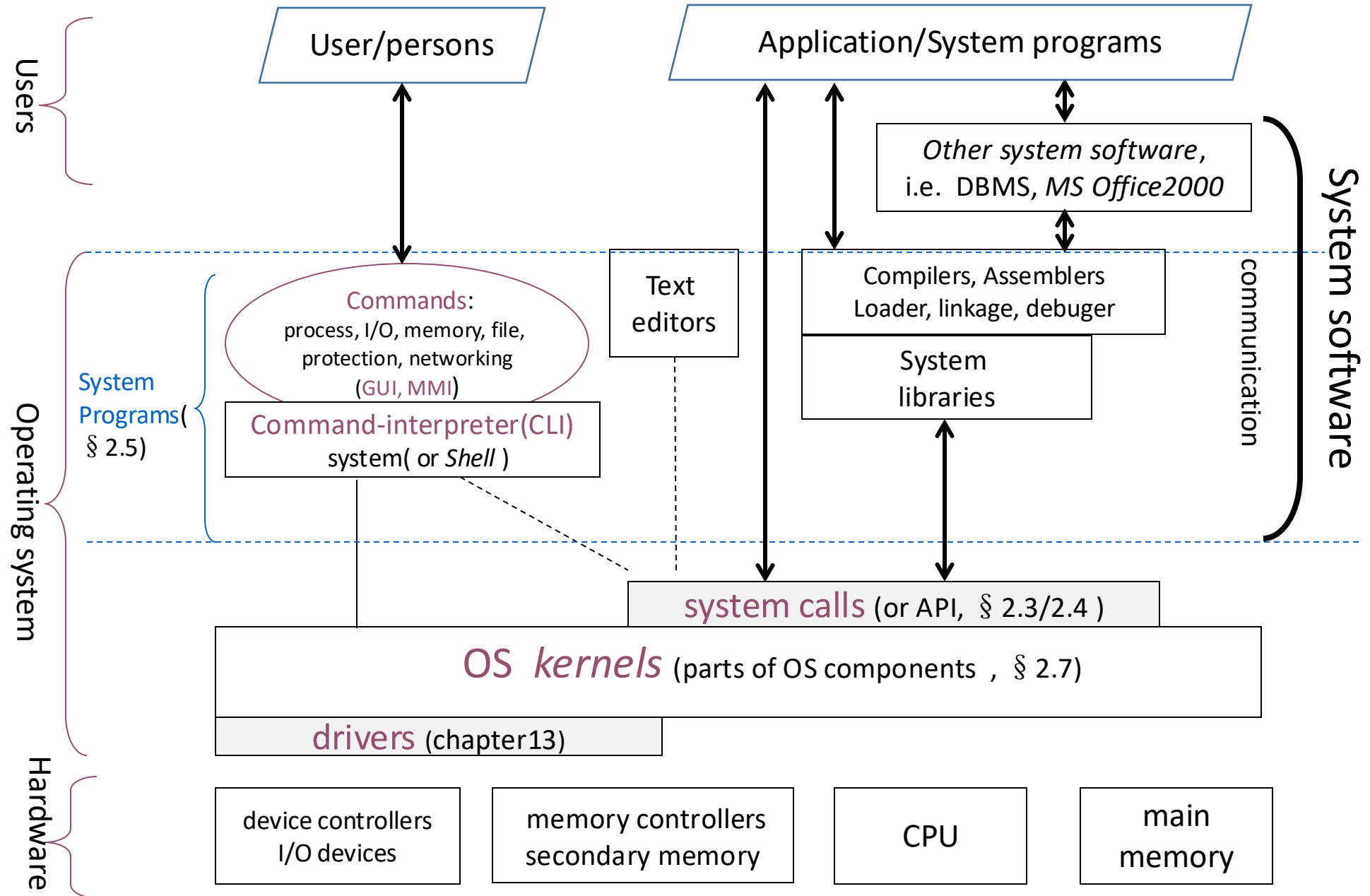
守护进程

运行在后台的一种特殊进程，独立于控制终端，并且周期性地执行某种任务或循环等待处理某些事件的发生；

无需用户输入就能运行，对整个系统或是对某个用户程序提供服务；

Linux系统的许多服务器通过守护进程实现；

守护进程一般在系统启动时开始运行，除非强行终止，否则直到系统关机才随之一起停止运行



2.6 System Design and Implementation

- Design and Implementation of OS not “solvable”, but some approaches have been proven successful
- Start the design by defining **goals** and **specifications**
- **User** goals and **System** goals
 - user goals – operating system should be **convenient to use**, easy to learn, reliable, safe, and fast
 - system goals – operating system should be **easy to design**, implement, and maintain, as well as flexible, reliable, error-free, and efficient

Operating System Design and Implementation (Cont.)

- Important principle to separate
 - **policy**: *What* will be done?
 - **mechanism**: *How* to do it?
- Mechanisms determine how to do something, policies decide what will be done
- The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later (example – timer)

Implementation

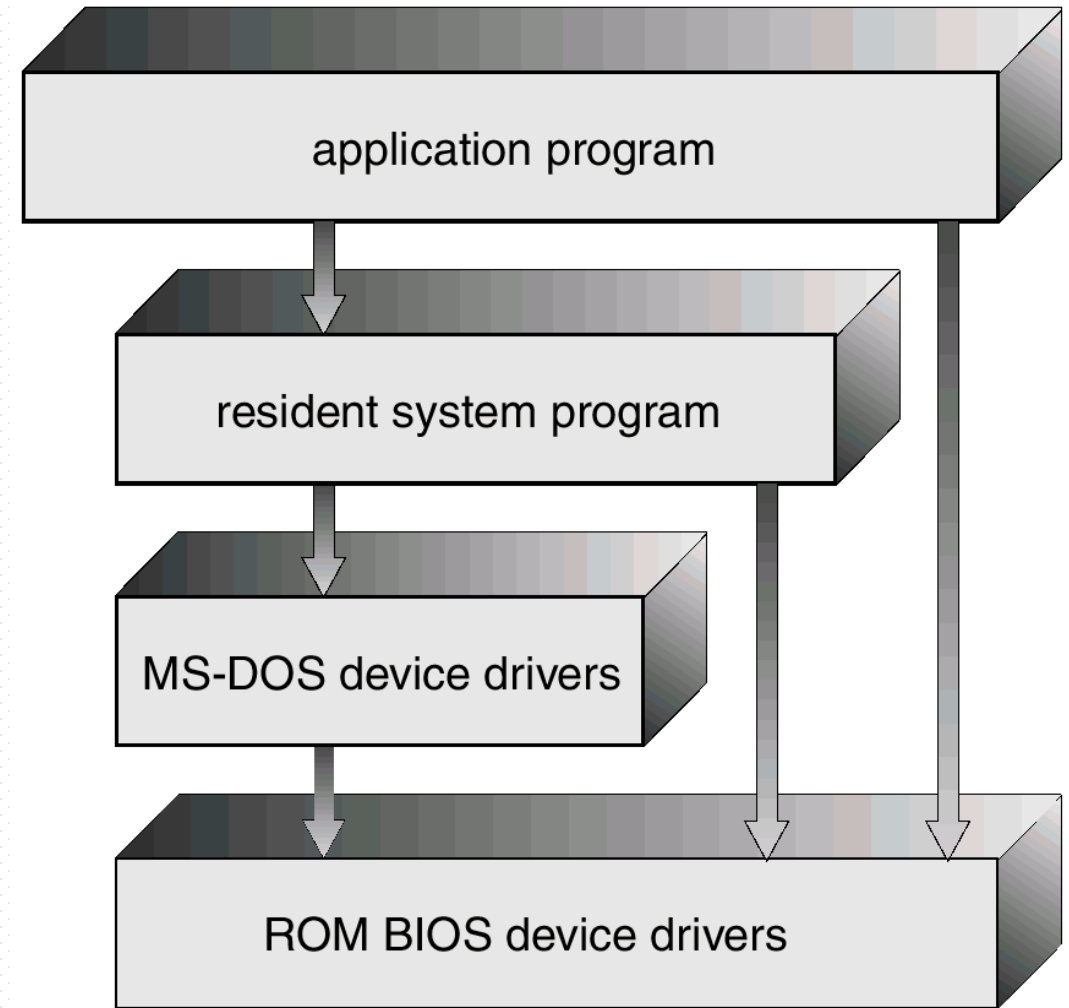
- Much variation
 - ▣ early OSes in assembly language
 - ▣ then system programming languages like Algol, PL/1
 - ▣ now C, C++
 - C++ is first invented for coding Unix kernel in Bell Lab during late 1960s
- Actually usually a mix of languages
 - ▣ lowest levels in assembly
 - ▣ main body in C
 - ▣ systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port/移植** to other hardware
 - ▣ But slower

Operating System Structure

- General-purpose OS is very large program
 - ▣ e.g. Win11安装包大概5GB左右，安装完后占用空间大约40GB
- Various ways to structure ones
 - ▣ simple structure – MS-DOS
 - ▣ more complex – UNIX
 - monolithic/macro kernel/宏内核
 - ▣ layered – an abstraction-OS2
 - ▣ microkernel –Mach/微内核
 - ▣ monolithic + modules – Linux, Solaris
 - ▣ hybrid system-iOS, Android

Simple Structure, ill-structured: MS-DOS

- For the introduction to DOS, refer to Appendix 2-2-1 ▶
- In view of architectures, MS-DOS aims to provide the most functionality in the least space
- Coded in *assembly language*, not divided into modules
- although MS-DOS has some structure, its interfaces and levels of functionality are not well separated
 - ▣ i.e. simple structure, ill-structured

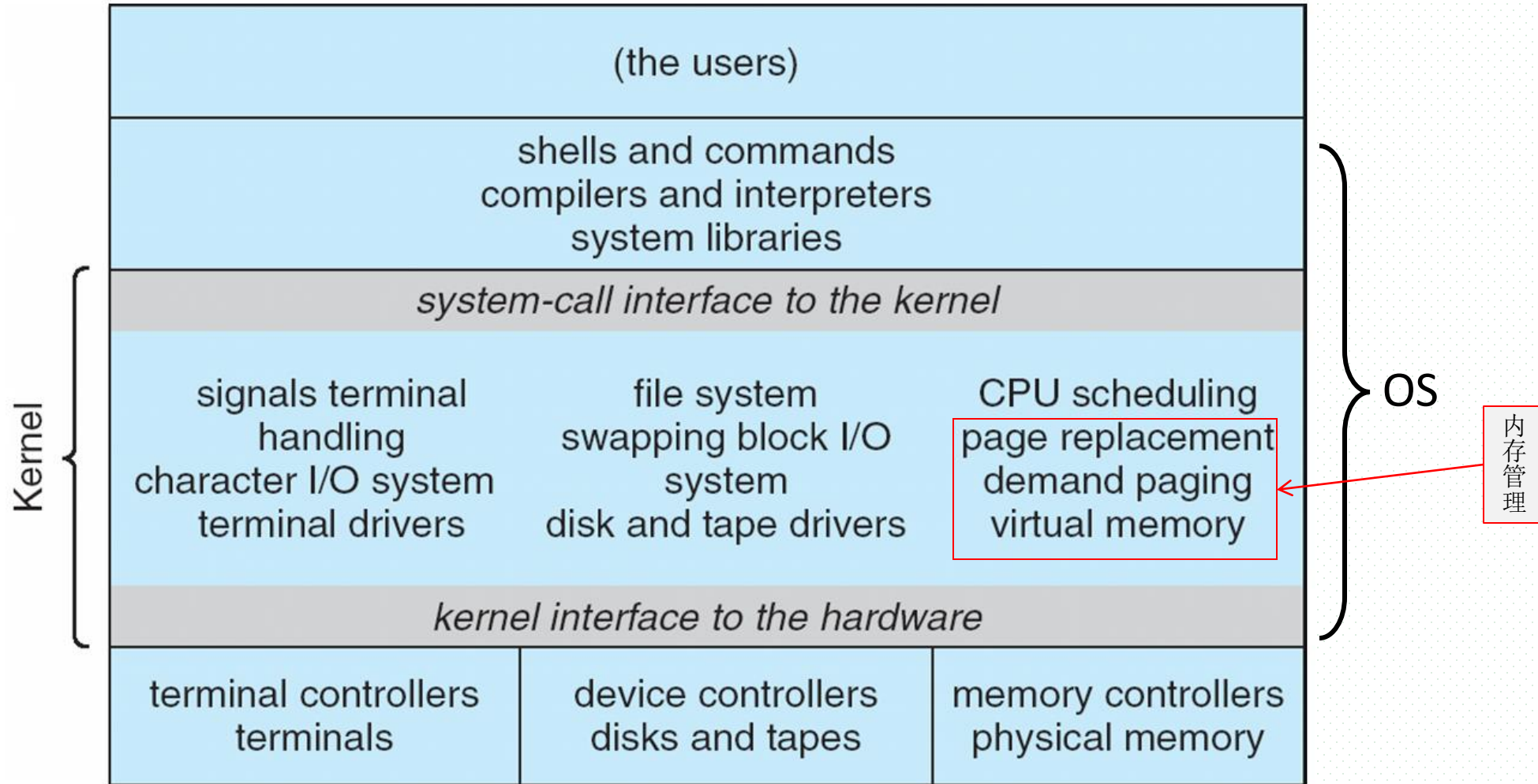


Simple Structure, monolithic/macro kernel (宏内核) : Unix

- For the introduction to Unix, refer to Appendix 2-2-2 
- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring
- The UNIX OS consists of two separable parts
 - ▣ systems programs
 - ▣ the kernel, running in kernel mode
 - consists of everything below the system-call interface and above the physical hardware
 - provides the file system, CPU scheduling, memory management, and other operating-system functions;
 - a large number of functions for one level

Traditional UNIX System Structure: kernel + system programs

- Beyond simple but not fully layered



Unix System Structure

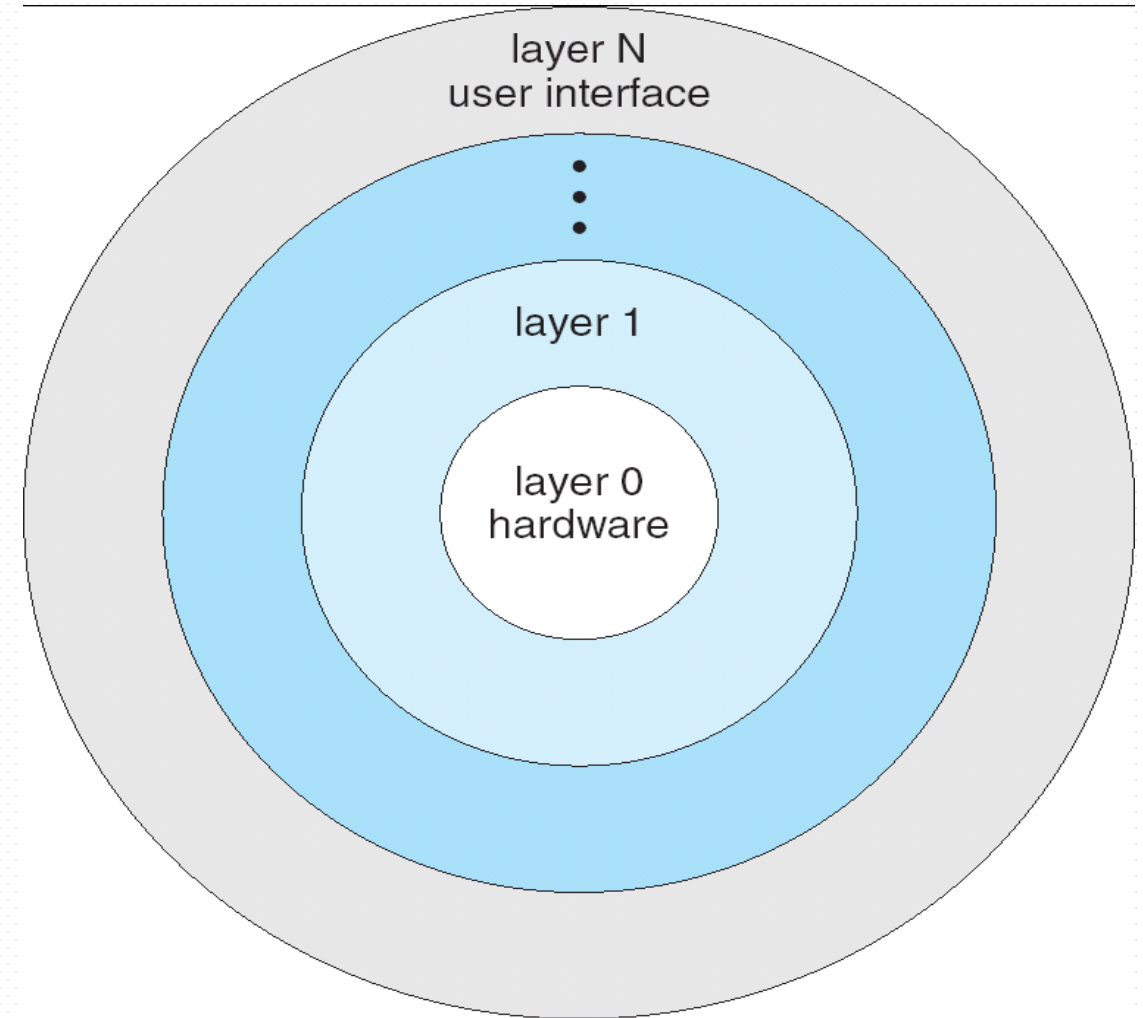
Simple Structure: Unix (Cont.)

- Demerits of Unix

- ❑ too many functions in kernel, i.e. too many OS components in one layer, and these functions or components may interference with each others
 - too big a kernel, couples among components
- ❑ big kernel makes UNIX difficult to enhance, as changes in one section could adversely affect other areas

Layered Approach: IBM OS2

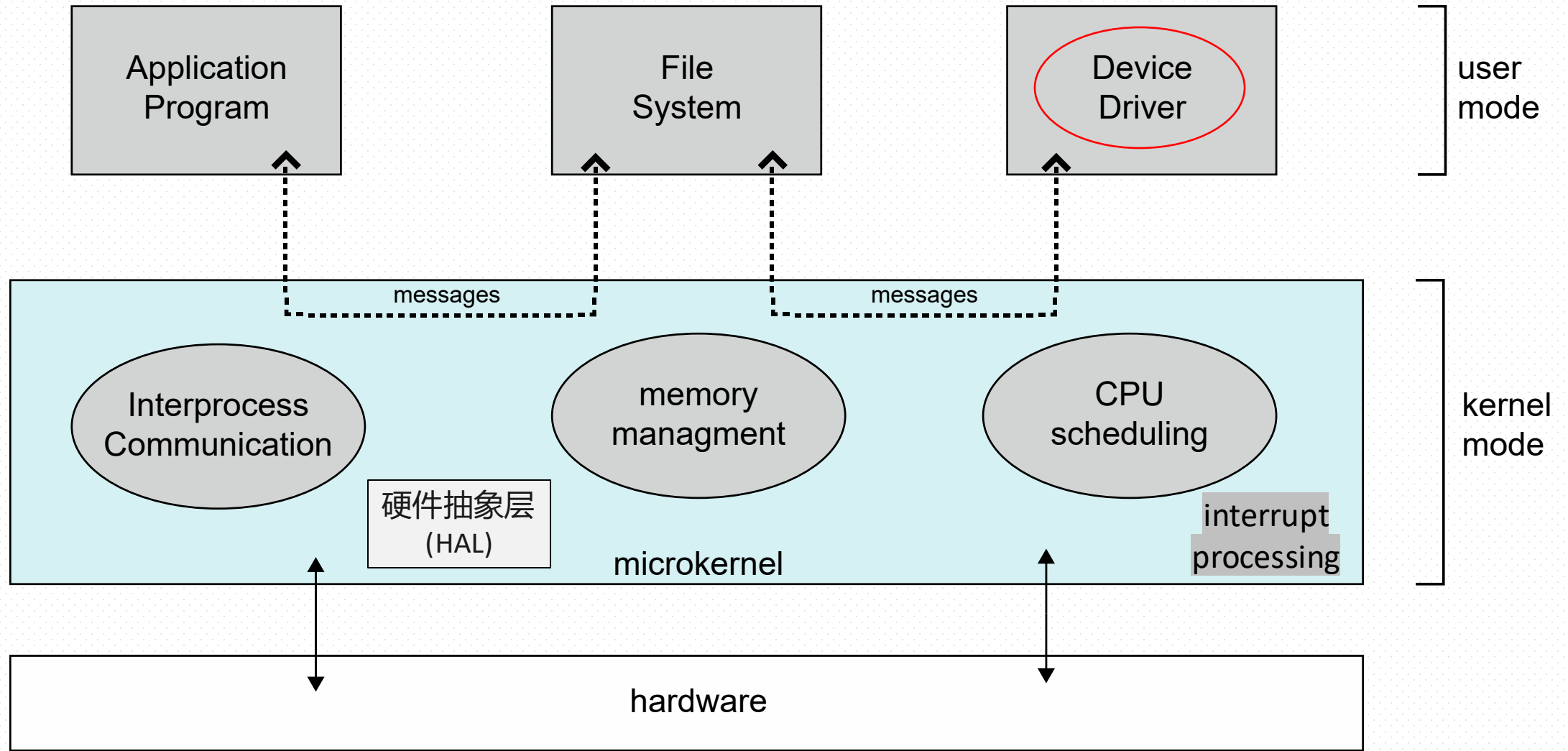
- The operating system is divided into a number of layers (levels), each built on top of lower layers
 - ▣ the bottom layer (layer 0), is the hardware
 - ▣ the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers
- E.g. IBM OS/2
 - ▣ Out of date, died
- Demerits
 - ▣ difficulties in layers definition, less efficient



Microkernel/微内核 System Structure: Mach

- OS is structured by removing all nonessential components from the kernel in the kernel mode, and implement them as system-level and user-level programs
 - ❑ moves as much from the kernel into “user” space/mode, thus result in a smaller kernel called **microkernel running in kernel mode**
 - ❑ the removed OS functional components run as **server processes in user mode**, and provide services such as file management services and driver services, etc
 - ❑ communication takes place between user modules using **message passing**
- Benefits:
 - ❑ easier to extend a microkernel//扩展性
 - ❑ easier to port the operating system to new architectures //易于移植
 - ❑ more reliable (less code is running in kernel mode) //可靠
 - ❑ more secure //安全
- Detriments
 - ❑ performance overhead of user space to kernel space communication//性能负荷
 - ❑ when a program in user space wishes to access a OS service, it indirectly interacts with the OS server process that provides this service in the same user space via exchanging message with the microkernel

Microkernel System Structure



Microkernel System Structure

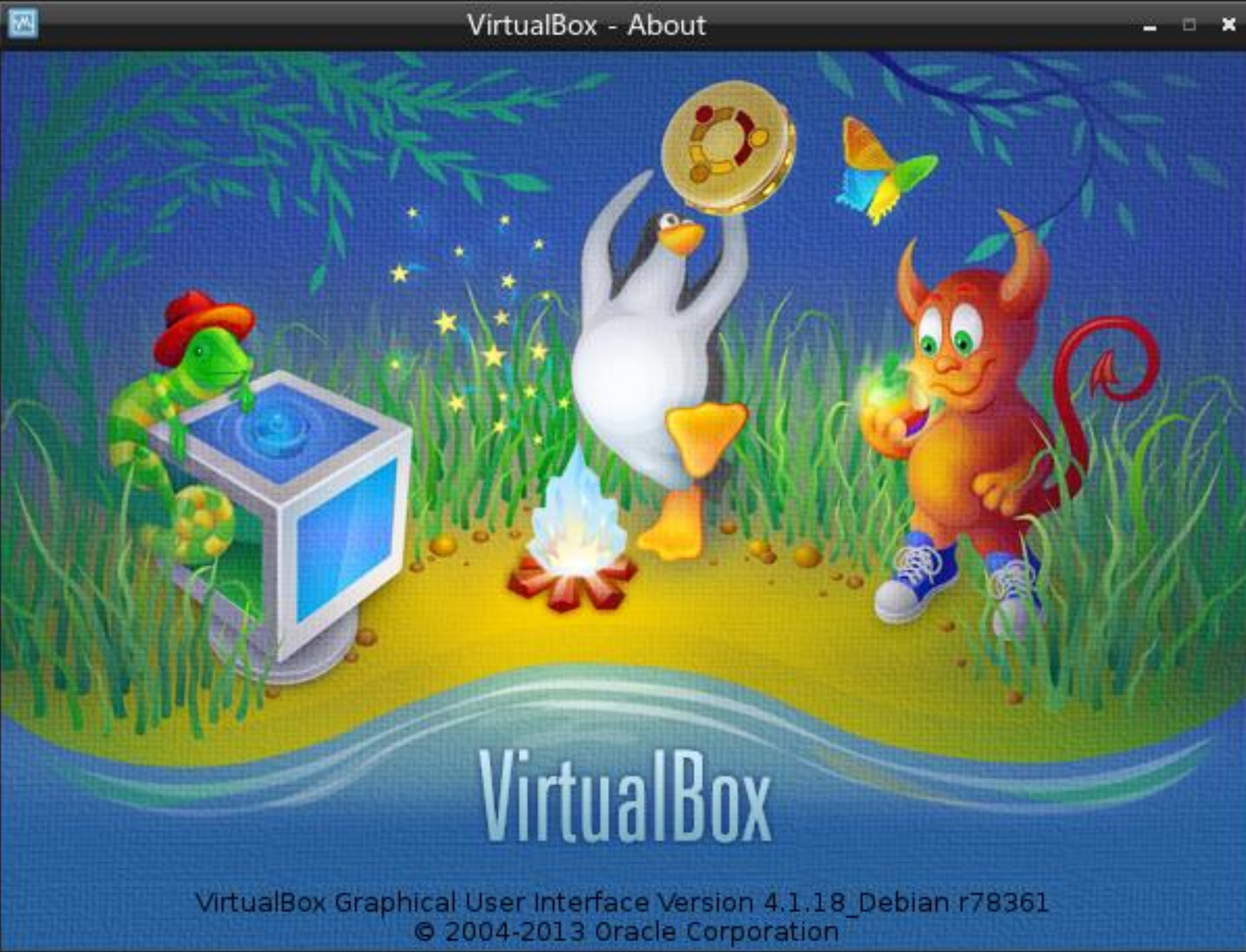
- OS = kernel + system programs
 - kernel = **microkernel** + OS server processes running in user mode
- **Mach**, example of **microkernel**
 - first microkernel-based OS, developed at CMU, in mid-1980s
 - Mac OS X kernel (**Darwin**) partly based on Mach
- Other microkernel-based OS
 - Tru64 Unix, QNX
 - Huawei Harmony

2.9 Operating System Generation

- Operating systems are designed to run on any of a class of machines; the system must be configured for each specific computer site
- **SYSGEN** program obtains information concerning the specific configuration of the hardware system
 - ▣ used to build system-specific compiled kernel or system-tuned
 - ▣ can generate more efficient code than one general kernel

System Boot (启动、引导, startup)

- When power initialized on system, execution starts at a fixed memory location
 - ▣ Firmware ROM used to hold initial boot code
- Operating system must be made available to hardware so hardware can start it
 - ▣ small piece of code – **bootstrap/引导程序 loader**, stored in **ROM** or **EEPROM** locates the kernel, loads it into **RAM**, and starts it
 - ▣ sometimes two-step process where **boot block/引导块** at fixed location loaded by ROM code, which loads bootstrap loader from disk
- Common bootstrap loader, **GRUB**, allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then **running**
- Two booting modes
 - ▣ BIOS
 - ▣ UEFI



Test

- 1. 与宏内核操作系统机比，下列特征中微内核操作系统具有的是（）
I. 较好的性能 II. 较高的可靠性 III. 较高的安全性 IV. 较强的可扩展性
A. 仅II, IV
B. 仅I, II, IV
C. 仅I, III, IV
D. 仅II, III, IV

- 2. 计算机开机后，操作系统最终被加载到（）
A. BIOS B. ROM C. EPROM D. RAM

- 3. 执行系统调用的过程包括以下指令，正确的顺序是（）
I. 返回用户态 II. 执行陷入指令 III. 传递系统调用参数 IV. 执行响应的服务程序
A. 2314
B. 2431
C. 3241
D. 3421



Thanks for your
attention



北京邮电大学